

DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 11:

Parallel Matrix Multiplication

What is dense linear algebra?

Not just matrix multiply!

- Linear systems: $Ax = b$
- Least squares: choose x to minimize $\|Ax - b\|_2^2$
 - Overdetermined or underdetermined; Unconstrained, constrained, weighted
- Eigenvalues: $Ax = \lambda x$ (or $Ax = \lambda Bx$)
- Matrix factorization
 - $A = LU$, $A = LL^T$
 - $A = QR$
 - $A = V\Lambda V^{-1}$, $A = QTQ^T$
 - $A = U\Sigma V^T$

A brief history of (Dense) Linear Algebra software

BLAS 1 were invented (1973–1977)

- Standard library of 15 ops (mostly) on vectors
 - Up to four versions of each: S/D/C/Z,), 46 routines, 3300 LOC
 - Example: DAXPY
 - Double precision (real)
 - Computes $Ax + y$
 - Goals
 - Common “pattern” to ease programming, readability
 - Robustness, via careful coding (avoiding over/underflow)
 - Portability + Efficiency via machine specific implementations
- BLAS 1 == $O(n^1)$ ops on $O(n^1)$ data
- Used in libraries like LINPACK
- Consider AXPY ($y = \alpha x + y$): $2n$ flops on $3n$ read/writes; so computational intensity is just $2/3$

A brief history of (Dense) Linear Algebra software

BLAS 2 were invented (1984–1986)

- Standard library of 25 ops (mostly) on matrix/vector pairs
 - Different data types and matrix types
 - Example: DGEMV
 - Double precision
 - GEneral matrix
 - Matrix-Vector product
- Goals
 - BLAS1 insufficient
 - BLAS2 for better vectorization (when vector machines roamed)
- BLAS2 == $O(n^2)$ ops on $O(n^2)$ data
- Computational intensity of AXPY is 2; still too low to run near peak speed

A brief history of (Dense) Linear Algebra software

BLAS 3 (1987–1988)

- Standard library of 9 ops (mostly) on matrix/matrix pairs
- Different data types and matrix types
- Example: DGEMM
 - Double precision
 - GEneral matrix
 - Matrix-Matrix product
 - Ex.: $C = \alpha AB + \beta C$, $C = \alpha AA^T + \beta C$, $B = T^{-1}B$
- BLAS3 == $O(n^3)$ ops on $O(n^2)$ data
- So computational intensity AXPY $(2n^3)/(4n^2) = n/2$
 - Big for efficient cache utilization
- Source: 142 routines, 31K LOC, Testing: 28K LOC
- Part of standard math libraries (eg AMD ACML, Intel MKL)
- <http://www.netlib.org/blas>

LAPACK: “Linear Algebra PACKage”

LAPACK (1989–now): <http://www.netlib.org/lapack>

- Uses BLAS-3
 - Parallel to the extent BLAS are parallel (on SMP)
 - Linear systems and least squares are nearly 100% BLAS 3
 - Eigenproblems, SVD — only about 50% BLAS 3
- Careful error bounds on everything
- Lots of variants depending on A 's structures

ScaLAPACK: “Scalable LAPACK”

ScaLAPACK (1995–present):

<http://www.netlib.org/scalapack>

- For distributed memory - uses MPI implementations
- More complex data structures, algorithms than LAPACK
- Only a small subset of LAPACK functionality

PLASMA and MAGMA (2008–now):

- Parallel LA Software for Multicore Architectures
 - Target: Shared memory multiprocessors
 - Stacks on LAPACK/BLAS interfaces
 - Tile algorithms, tile data layout, dynamic scheduling
 - Other algorithmic ideas, too (randomization, etc)
- Matrix Algebra for GPU and Multicore Architectures
 - Target: CUDA, OpenCL, Xeon Phi
 - Still stacks (e.g. on CUDA BLAS)
 - Again: tile algorithms + data, dynamic scheduling
 - Mixed precision algorithms (+ iterative refinement)
- Dist memory: DPLASMA

LAPACK routine types

- Linear systems (general, symmetric, SPD)
- Least squares (overdetermined, underdetermined, constrained, weighted)
- Symmetric eigenvalues and vectors
 - Standard: $Ax = \lambda x$
 - Generalized: $Ax = \lambda Bx$
- Nonsymmetric eigenproblems
 - Schur form: $A = QTQ^T$
 - Eigenvalues/vectors
 - Invariant subspaces
 - Generalized variants
- SVD (standard/generalized)
- Different interfaces
 - Simple drivers
 - Expert drivers with error bounds, extra precision, etc
 - Low-level routines

Matrix vector product

Simple $y = Ax$ involves two indices

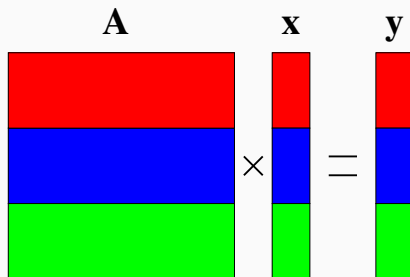
$$y_i = \sum_j A_{ij} x_j$$

Can organize around either one:

```
1 % Row-oriented
2 for i = 1:n
3     y(i) = A(i,:) * x;
4 end
5
6 % Col-oriented
7 y = 0;
8 for j = 1:n
9     y = y + A(:,j) * x(j);
10 end
```

... or deal with index space in other ways!

Parallel matvec: 1D row-blocked



Receive broadcast x_0, x_1, x_2 into local x_0, x_1, x_2 ; then

$$\text{On P0: } A_{00}x_0 + A_{01}x_1 + A_{02}x_2 = y_0$$

$$\text{On P1: } A_{10}x_0 + A_{11}x_1 + A_{12}x_2 = y_1$$

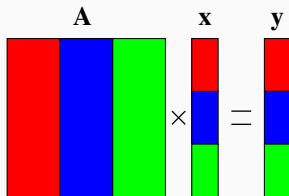
$$\text{On P2: } A_{20}x_0 + A_{21}x_1 + A_{22}x_2 = y_2$$

Algorithm uses the formula

$$y(i) = y(i) + A(i) * x = y(i) + \sum_j A(i, j) * x(j)$$

Parallel matvec: 1D col-blocked

A column layout of the matrix eliminates the broadcast of x but adds a reduction to update the destination y :



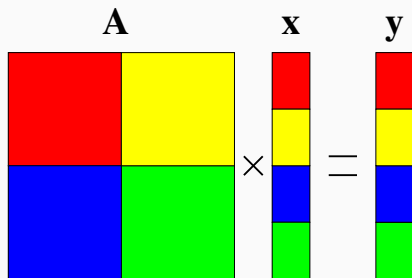
Independently compute

$$z^{(0)} = \begin{bmatrix} A_{00} \\ A_{10} \\ A_{20} \end{bmatrix} x_0 \quad z^{(1)} = \begin{bmatrix} A_{00} \\ A_{10} \\ A_{20} \end{bmatrix} x_1 \quad z^{(2)} = \begin{bmatrix} A_{00} \\ A_{10} \\ A_{20} \end{bmatrix} x_2$$

and perform reduction: $y = z^{(0)} + z^{(1)} + z^{(2)}$.

Parallel matvec: 2D blocked

A 2D blocked layout uses a broadcast and reduction, both on a subset of processors ($\sqrt{p}$ for square processor grid)



- Involves broadcast *and* reduction
- ... but with subsets of processors

Parallel matvec: 2D blocked

Broadcast x_0, x_1 to local copies x_0, x_1 at **P0** and **P2**

Broadcast x_2, x_3 to local copies x_2, x_3 at **P1** and **P3**

In parallel, compute

$$\begin{aligned} \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} &= \begin{bmatrix} z_0^{(0)} \\ z_1^{(0)} \end{bmatrix} & \begin{bmatrix} A_{02} & A_{03} \\ A_{12} & A_{13} \end{bmatrix} \begin{bmatrix} x_2 \\ x_3 \end{bmatrix} &= \begin{bmatrix} z_0^{(1)} \\ z_1^{(1)} \end{bmatrix} \\ \begin{bmatrix} A_{20} & A_{21} \\ A_{30} & A_{31} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} &= \begin{bmatrix} z_2^{(3)} \\ z_3^{(3)} \end{bmatrix} & \begin{bmatrix} A_{20} & A_{21} \\ A_{30} & A_{31} \end{bmatrix} \begin{bmatrix} x_2 \\ x_3 \end{bmatrix} &= \begin{bmatrix} z_2^{(3)} \\ z_3^{(3)} \end{bmatrix} \end{aligned}$$

Reduce across rows:

$$\begin{bmatrix} y_0 \\ y_1 \end{bmatrix} = \begin{bmatrix} z_0^{(0)} \\ z_1^{(0)} \end{bmatrix} + \begin{bmatrix} z_0^{(1)} \\ z_1^{(1)} \end{bmatrix} \qquad \begin{bmatrix} y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} z_2^{(2)} \\ z_3^{(2)} \end{bmatrix} + \begin{bmatrix} z_2^{(3)} \\ z_3^{(3)} \end{bmatrix}$$

Parallel Matrix Multiplication

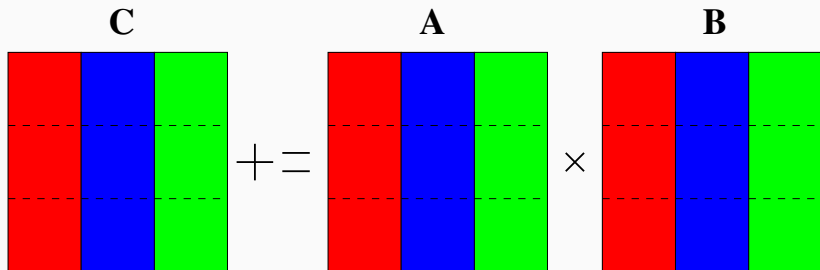
The performance depends on several factors:

- The interconnection network. Networks with higher connectivity, like the hypercube and mesh, permit faster algorithms than networks like rings and busses.
- The data layout, or where A, B and C are stored on the processors.
- The algorithm.
 - Basic operation: $C = C + AB$
 - Computation: $2n^3$ flops
 - Goal: $2n^3/p$ flops per processor, minimal communication

Use of performance models for algorithm design:

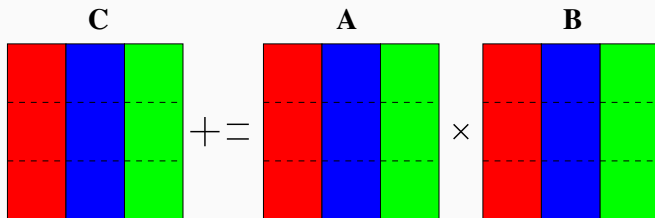
Message Time = latency + #words * time-per-word = $\alpha + n * \beta$

1D Column Blocked Layout



- $A(:, j)$ means j th block column
- Processor j owns $A(:, j)$, $B(:, j)$, $C(:, j)$
- $C(:, j)$ depends on *all* of A , but only $B(:, j)$,
 $C(:, j) = C(:, j) + A * B(:, j)$
- Only A needs to be communicated ; how do we communicate pieces of A ?

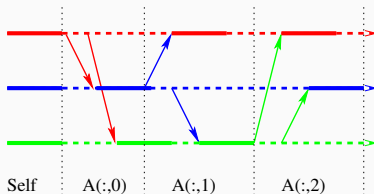
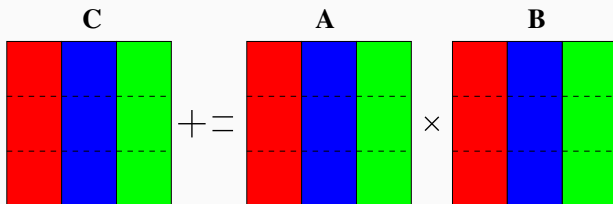
1D layout on bus (no broadcast)



- Everyone computes local contributions first
- **P0** sends $A(:, 0)$ to each processor j in turn; processor j receives, computes $A(:, 0)B(0, j)$
- **P1** sends $A(:, 1)$ to each processor j in turn; processor j receives, computes $A(:, 1)B(1, j)$
- **P2** sends $A(:, 2)$ to each processor j in turn; processor j receives, computes $A(:, 2)B(2, j)$

1D layout on bus (no broadcast)

First consider a bus-connected machine without broadcast:
only one pair of processors can communicate at a time
(ethernet)



1D layout on bus (no broadcast)

```
1 C(:,myproc) += A(:,myproc)*B(myproc,myproc)
2 for i = 0:p-1
3     for j = 0:p-1, except i
4         if (myproc == i)
5             send A(:,i) to processor j
6         if (myproc == j)
7             receive A(:,i) from i
8             C(:,myproc) += A(:,i)*B(i,myproc)
9     barrier
```

Performance model: 1D layout on bus (no broadcast)

Only 1 pair of processors (i and j) are active on any iteration, and of those, only i is doing computation; no overlapping communications, so in a simple $\alpha - \beta$ model:

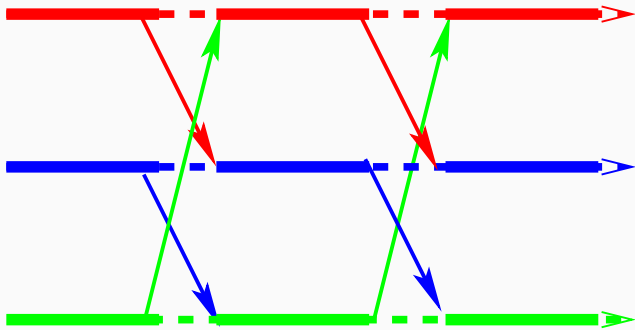
- Computations: $2n(n/p)^2 = 2n^3/p^2$
- Messages: $p(p - 1)$
- Data: each message involves n^2/p words
- Communication cost: $p(p - 1)\alpha + (p - 1)n^2\beta$

Running time $\approx 2n^3 + p^2\alpha + pn^2\beta$

This is worse than the serial time and grows with p . Useful only for big problem that doesn't fit on one processor.

1D layout on a Processor Ring

Pairs of adjacent processors can communicate simultaneously.



- Every process j can send data to $j + 1$ simultaneously
- Pass slices of A around the ring until everyone sees the whole matrix ($p - 1$ phases).

1D layout on ring

```
1 tmp = A(myproc)
2 C(myproc) += tmp*B(myproc,myproc)
3 for j = 1 to p-1
4     sendrecv tmp to myproc+1 mod p,
5             from myproc-1 mod p
6     C(myproc) += tmp*B(myproc-j mod p, myproc)
```

Performance model: 1D layout on ring

In a simple $\alpha - \beta$ model, at each processor:

- $p - 1$ message sends (and simultaneous receives)
- Each message involves n^2/p data
- Communication cost: $(p - 1)\alpha + (1 - 1/p)n^2\beta$

Running time $\approx 2n^3/p + 2p\alpha + 2n^2\beta$

Parallel efficiency grows to 1 as n/p increases (or as α, β get smaller)

Need to try 2D Matrix layout

Outer product algorithm

Serial: Recall outer product organization:

```
1 for k = 0:s-1
2   C += A(:,k)*B(k,:);
3 end
```

Parallel: Assume $p = s^2$ processors, block $s \times s$ matrices.

For a 2×2 example:

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00}B_{00} & A_{00}B_{01} \\ A_{10}B_{00} & A_{10}B_{01} \end{bmatrix} + \begin{bmatrix} A_{01}B_{10} & A_{01}B_{11} \\ A_{11}B_{10} & A_{11}B_{11} \end{bmatrix}$$

- Processor for each $(i, j) \implies$ parallel work for each k !
- Note everyone in row i uses $A(i, k)$ at once, and everyone in row j uses $B(k, j)$ at once.

Parallel outer product (SUMMA)

```
1  for k = 0:s-1
2    for each i in parallel
3      broadcast A(i,k) to row
4    for each j in parallel
5      broadcast A(k,j) to col
6    On processor (i,j), C(i,j) += A(i,k)*B(k,j);
7  end
```

If we have tree along each row/column, then

- $\log(s)$ messages per broadcast
- $\alpha + \beta n^2/s^2$ per message
- $2\log(s)(\alpha s + \beta n^2/s)$ total communication
- Compare to 1D ring: $(p-1)\alpha + (1-1/p)n^2\beta$

Note: Same ideas work with block size $b < n/s$

Cannon's algorithm

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00}B_{00} & A_{01}B_{11} \\ A_{11}B_{10} & A_{10}B_{01} \end{bmatrix} + \begin{bmatrix} A_{01}B_{10} & A_{00}B_{01} \\ A_{10}B_{00} & A_{11}B_{11} \end{bmatrix}$$

Idea: Reindex products in block matrix multiply

$$\begin{aligned} C(i, j) &= \sum_{k=0}^{p-1} A(i, k) B(k, j) \\ &= \sum_{k=0}^{p-1} A(i, k + i + j \bmod p) B(k + i + j \bmod p, j) \end{aligned}$$

For a fixed k , a given block of A (or B) is needed for contribution to *exactly one* $C(i, j)$.

Cannon's algorithm

```
1  % Move A(i,j) to A(i,i+j)
2  for i = 0 to s-1
3      cycle A(i,:) left by i
4
5  % Move B(i,j) to B(i+j,j)
6  for j = 0 to s-1
7      cycle B(:,j) up by j
8
9  for k = 0 to s-1
10     in parallel;
11         C(i,j) = C(i,j) + A(i,j)*B(i,j);
12     cycle A(:,i) left by 1
13     cycle B(:,j) up by 1
```

Cost of Cannon

- Assume 2D torus topology
- Initial cyclic shifts: $\leq s$ messages each ($\leq 2s$ total)
- For each phase: 2 messages each ($2s$ total)
- Each message is size n^2/s^2
- Communication cost: $4s(\alpha + \beta n^2/s^2) = 4(\alpha s + \beta n^2/s)$
- This communication cost is optimal!
... but SUMMA is simpler, more flexible, almost as good

Speedup and efficiency

Recall

$$\text{Speedup} := t_{\text{serial}} / t_{\text{parallel}}$$

$$\text{Efficiency} := \text{Speedup} / p$$

Assuming no overlap of communication and computation, efficiencies are

1D layout	$\left(1 + O\left(\frac{p}{n}\right)\right)^{-1}$
SUMMA	$\left(1 + O\left(\frac{\sqrt{p} \log p}{n}\right)\right)^{-1}$
Cannon	$\left(1 + O\left(\frac{\sqrt{p}}{n}\right)\right)^{-1}$

- Basic operation: $C = C + AB$
- Computation: $2n^3$ flops
- Goal: $2n^3/p$ flops per processor, minimal communication
- Two main contenders: SUMMA and Cannon

Outer product algorithm

Serial: Recall outer product organization:

```
1 for k = 0:s-1
2   C += A(:,k)*B(k,:);
3 end
```

Parallel: Assume $p = s^2$ processors, block $s \times s$ matrices.

For a 2×2 example:

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00}B_{00} & A_{00}B_{01} \\ A_{10}B_{00} & A_{10}B_{01} \end{bmatrix} + \begin{bmatrix} A_{01}B_{10} & A_{01}B_{11} \\ A_{11}B_{10} & A_{11}B_{11} \end{bmatrix}$$

- Processor for each $(i, j) \implies$ parallel work for each k !
- Note everyone in row i uses $A(i, k)$ at once, and everyone in row j uses $B(k, j)$ at once.

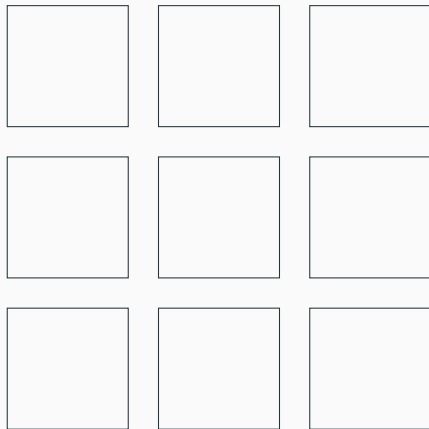
Parallel outer product (SUMMA)

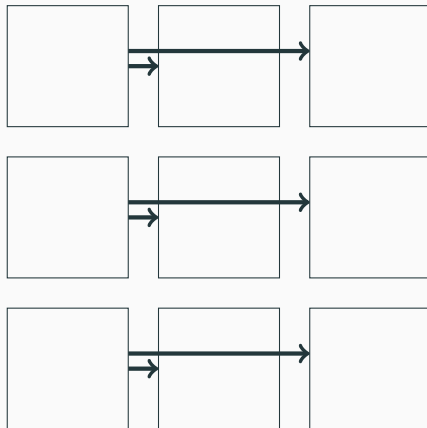
```
1  for k = 0:s-1
2    for each i in parallel
3      broadcast A(i,k) to row
4    for each j in parallel
5      broadcast A(k,j) to col
6    On processor (i,j), C(i,j) += A(i,k)*B(k,j);
7  end
```

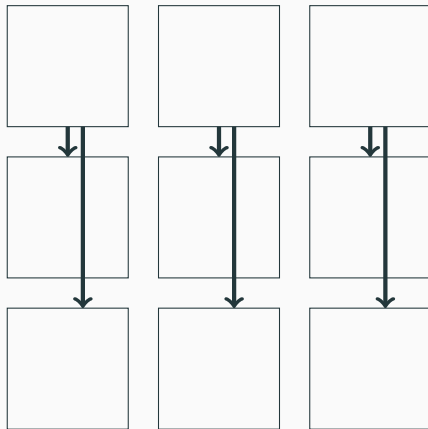
If we have tree along each row/column, then

- $\log(s)$ messages per broadcast
- $\alpha + \beta n^2/s^2$ per message
- $2\log(s)(\alpha s + \beta n^2/s)$ total communication
- Compare to 1D ring: $(p-1)\alpha + (1-1/p)n^2\beta$

Note: Same ideas work with block size $b < n/s$







Parallel outer product (SUMMA)

If we have tree along each row/column, then

- $\log(s)$ messages per broadcast
- $\alpha + \beta n^2/s^2$ per message
- $2 \log(s)(\alpha s + \beta n^2/s)$ total communication

Assuming communication and computation can potentially overlap *completely*, what does the speedup curve look like?

Cannon's algorithm

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00}B_{00} & A_{01}B_{11} \\ A_{11}B_{10} & A_{10}B_{01} \end{bmatrix} + \begin{bmatrix} A_{01}B_{10} & A_{00}B_{01} \\ A_{10}B_{00} & A_{11}B_{11} \end{bmatrix}$$

Idea: Reindex products in block matrix multiply

$$\begin{aligned} C(i, j) &= \sum_{k=0}^{p-1} A(i, k) B(k, j) \\ &= \sum_{k=0}^{p-1} A(i, k + i + j \bmod p) B(k + i + j \bmod p, j) \end{aligned}$$

For a fixed k , a given block of A (or B) is needed for contribution to *exactly one* $C(i, j)$.

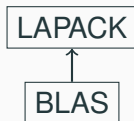
Cannon's algorithm

```
1  % Move A(i,j) to A(i,i+j)
2  for i = 0 to s-1
3      cycle A(i,:) left by i
4
5  % Move B(i,j) to B(i+j,j)
6  for j = 0 to s-1
7      cycle B(:,j) up by j
8
9  for k = 0 to s-1
10     in parallel;
11         C(i,j) = C(i,j) + A(i,j)*B(i,j);
12     cycle A(:,i) left by 1
13     cycle B(:,j) up by 1
```

Cost of Cannon

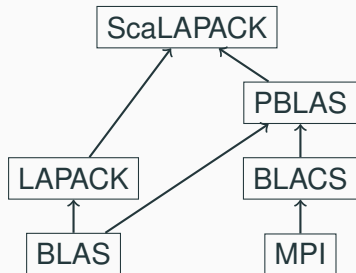
- Assume 2D torus topology
- Initial cyclic shifts: $\leq s$ messages each ($\leq 2s$ total)
- For each phase: 2 messages each ($2s$ total)
- Each message is size n^2/s^2
- Communication cost: $4s(\alpha + \beta n^2/s^2) = 4(\alpha s + \beta n^2/s)$
- This communication cost is optimal!
... but SUMMA is simpler, more flexible, almost as good

Reminder: Why matrix multiply?



Build fast serial linear algebra (LAPACK) on top of BLAS 3.

Reminder: Why matrix multiply?



ScaLAPACK builds additional layers on same idea.